



**FACULTY OF TECHNOLOGY AND ENGINEERING
DEVANG PATEL INSTITUTE OF ADVANCE TECHNOLOGY
AND RESEARCH**

DEPARTMENT OF INFORMATION TECHNOLOGY

A.Y. 2026-27 Semester-3

LAB MANUAL

CSUC201 Fundamentals of Data Structures and Algorithms



PRACTICAL-1

Practical AIM:

To implement fundamental array operations.

1.1

Problem Definition:

A bakery prepares n items every morning and places them in a display row. At the end of each hour, the leftmost item is moved to the rightmost position to make room for fresh stock at the front. Given the initial row and the number of hours h , print the final display order. Describe the approach you used. How does it behave when h is very large, say 10 million? Is there a way to get the correct result without performing the operation h times?

Output:

```
PS D:\DSA\Practical1> cd 'd:\DSA\Practical1\output'
PS D:\DSA\Practical1\output> & .\'1.1.exe'
Enter the number of items: 4
Enter the items: 123 456 789 322
Enter no.of hours: 5
456      789      322      123
```

Difficulty Faced:

The main difficulty was understanding how to move the first element to the last position repeatedly. When h was very large, performing the rotation one by one would take too much time.

Solution:

Used $h \% n$ to calculate the number of actual rotations required. Since the array becomes the same after every n rotations, this reduces the number of operations and makes the solution faster.

Outcome:

Successfully implemented the array rotation and optimized the solution using $h \% n$. The program can efficiently handle very large values of h without performing the rotation millions of times.

1.2

Problem Definition:

A library issues books to students and records each book's ID every time it is borrowed. At the end of the month, the librarian wants to find all books that were borrowed more than once, as those need priority restocking. Given the borrowing log, print all such book IDs.

Describe the approach you used. How many times does your solution scan the data? If the library had 100,000 borrow records, would your approach still be practical? Can you think of a way that requires fewer passes?

Output:

```
PS D:\DSA\Practical1\output> & .\'1.2.exe'  
Enter the number of books: 4  
Enter the IDs of the books: 532 567 431 532  
532
```

Difficulty Faced:

The main difficulty was finding which book IDs appeared more than once without repeatedly checking the entire borrowing log. With a large number of records, repeated scanning would make the program slower.

Solution:

Used a frequency-counting approach to keep track of how many times each book ID occurs. The data can be scanned once using a map, and IDs whose count is greater than 1 are printed. This is practical even for **100,000 records** and requires only **one pass** through the data.

Outcome:

Successfully identified all book IDs that were borrowed more than once. The optimized approach scans the borrowing log only once, making it practical even for 100,000 records.

1.3

Problem Definition:

A school newspaper editor wants to highlight the most impressive word from a submitted article on the front page. The word is chosen simply by length — the longest one wins. Given a sentence, print the winning word and how many letters it has.

Describe the approach you used. What does your solution do when two words have the same length? Is that behavior intentional, and can you think of a way to handle it explicitly?

Output:

```
PS D:\DSA\Practical1\output> & .\'1.3.exe'  
Enter a sentence: Hello World  
The longest word is: Hello  
Length of the longest word is: 5
```

Difficulty Faced:

The main difficulty was finding the longest word from the sentence while comparing the length of each word. Handling two words having the same maximum length was also a little confusing.

Solution:

Split the sentence into individual words and compare their lengths one by one. Whenever a longer word is found, store it as the answer. If two words have the same length, the solution keeps the **first word** found. This can be changed explicitly to keep the last word or print all words having the maximum length.

Outcome:

Successfully identified and displayed the longest word along with its length. When two words have the same maximum length, the program selects the first occurrence. This behavior can also be changed explicitly to display all words having the maximum length if required.

Key Questions:

1. [Problem 1] The display order repeats itself after a certain number of hours regardless of how large h gets. Without trying any specific example, explain why this repetition is inevitable and what determines when it first repeats.

Sol: The display repeats after n rotations, because every item returns to its original position. The first repetition is determined by the number of items (n).

2. [Problem 2] If the same book ID appears three times in the log, should it appear once or three times in your output? Does your solution handle this correctly, and how would you verify it?

Sol: If a book ID appears three times, it should appear only once in the output. The solution handles this using a frequency count. We can verify it by testing an ID that occurs three or more times.

3. [Problem 3] The editor later decides that punctuation attached to a word (e.g. "hello," or "great!") should not count toward its length. How would this change your solution, and at which step would you handle it?

Sol: Remove punctuation from each word before calculating its length. This should be handled after splitting the sentence into words and before comparing their lengths.

Github Link:

<https://github.com/brindajivani/DSA/tree/main/Practical1>